

Lab 4. Pandas

```
In [1]: import pandas as pd
import numpy as np
```

Task 1. (vowel and values above 50)

Create a Series of 8 random integers between 10 and 100 using `np.random.randint`.

- Label the index with the first 8 letters of the alphabet.
- Select only the elements whose index is a vowel and whose value is above 50 — all in one line.

```
In [2]: data=np.random.randint(10, 101, size=8)
s_data=pd.Series(data, index=["a","b","c","d","e","f","g","h"])
s_data[s_data.index.isin(["a","e"]) & (s_data>50)]
```

```
Out[2]: a    65
dtype: int64
```

Task 2. (operations on a DataFrame)

Generate a DataFrame of 6 rows × 4 columns of random integers between 1 and 100 using `np.random.randint` with indices `dates = pd.date_range("2024-01-01", periods=6)` columns `["A", "B", "C", "D"]`.

- Select rows where column "A" > 50 and "C" < 30 simultaneously.
- Add a column "E" which is the row sum of columns A-D.
- For rows where "B" is even, multiply "D" by 2 (use `.loc` conditional indexing).
- Reorder the columns to ["D", "A", "B", "C", "E"].
- Sort the DataFrame by column "E" in descending order.

```
In [3]: data_2=np.random.randint(1, 101, size=24).reshape(6,4)
dates=pd.date_range("2024-01-01", periods=6)
col=["A", "B", "C", "D"]
```

```
frame=pd.DataFrame(data_2,index=dates, columns=col)

#a)
print(frame[(frame["A"]>50) & (frame["C"]<30)])

#b)
frame["E"]=frame.sum(axis=1)
print(frame)

#c)
frame.loc[frame["B"]%2==0,"D"]*=2
print(frame)

#d)
frame = frame.loc[:, ["D", "A", "B", "C", "E"]]
print(frame)

#e)
frame = frame.sort_values(by="E", ascending=False)
print(frame)
```

	A	B	C	D	
2024-01-01	92	50	24	68	
	A	B	C	D	E
2024-01-01	92	50	24	68	234
2024-01-02	36	5	57	35	133
2024-01-03	27	37	85	60	209
2024-01-04	21	35	86	40	182
2024-01-05	74	44	79	67	264
2024-01-06	70	87	64	85	306
	A	B	C	D	E
2024-01-01	92	50	24	136	234
2024-01-02	36	5	57	35	133
2024-01-03	27	37	85	60	209
2024-01-04	21	35	86	40	182
2024-01-05	74	44	79	134	264
2024-01-06	70	87	64	85	306
	D	A	B	C	E
2024-01-01	136	92	50	24	234
2024-01-02	35	36	5	57	133
2024-01-03	60	27	37	85	209
2024-01-04	40	21	35	86	182
2024-01-05	134	74	44	79	264
2024-01-06	85	70	87	64	306
	D	A	B	C	E
2024-01-06	85	70	87	64	306
2024-01-05	134	74	44	79	264
2024-01-01	136	92	50	24	234
2024-01-03	60	27	37	85	209
2024-01-04	40	21	35	86	182
2024-01-02	35	36	5	57	133

Task 3. (stock prices)

Consider the data

```
data = {
    "day": range(1,11),
    "X": [100, 102, 101, None, 105, 107, 108, None, 110, 112],
    "Y": [50, 51, None, 53, 54, 55, 56, 57, None, 59],
    "Z": [200, 198, 202, 205, None, 210, 212, 215, 218, None]
}
df = pd.DataFrame(data)
```

where each column `X`, `Y`, and `Z` corresponds to stock prices. Below, by X_t we denote stock price of asset `X` on the t -th day, and analogously for Y_t and Z_t .

Perform the following tasks:

- Replace all missing stock prices with the mean of their respective column.
- Compute daily portfolio value P_t using the formula

$$P_t = w_X X_t + w_Y Y_t + w_Z Z_t,$$

where weights are $w_X = 0.4$, $w_Y = 0.3$, $w_Z = 0.3$.

- Use percent change day-over-day r_t (see function `pct_change()`) for both individual stocks and the portfolio:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

- Calculate the standard deviation of daily returns for each stock `X`, `Y`, `Z` and report the stock with the highest standard deviation.
- Create a flag for positive ($r_t > 0$) vs. negative portfolio ($r_t < 0$) returns. Group by this flag and compute the average portfolio value for each group.
- Prepare a table containing: `day`, `portfolio_value` and `portfolio_return`. Save the table to a CSV file named `portfolio_summary.csv`.

```
In [14]: data = {
    "day":      range(1,11),
    "X": [100, 102, 101, None, 105, 107, 108, None, 110, 112],
    "Y": [50, 51, None, 53, 54, 55, 56, 57, None, 59],
    "Z": [200, 198, 202, 205, None, 210, 212, 215, 218, None]
}
df = pd.DataFrame(data)

#a)
df=df.fillna(df.mean())

#b)
w_x=0.4
w_y=0.3
w_z=0.3

P=w_x*df["X"]+w_y*df["Y"]+w_z*df["Z"]
print(f"P:\n{P}")

#c)
r=P.pct_change()
```

```

print(f"\nr:\n{r}")
#d)

x_std=df["X"].pct_change().std()
y_std=df["Y"].pct_change().std()
z_std=df["Z"].pct_change().std()
m=np.argmax([x_std, y_std, z_std])
print(f"\n{df.columns[m+1]} has the biggest standard deviation: {round(max(x_std, y_std, z_std),6)}.")

#e)
flag=r[1:]>0    #no need to flag the 0-th NaN argument
grouped = P[1:].groupby(flag)
print(grouped.mean())

#f)
new_data={
    "day": data["day"],
    "portfolio_value": P,
    "portfolio_return": r
}

new_df=pd.DataFrame(new_data)
print(f"\nFinal dataframe:\n{new_df}")
new_df.to_csv("portfolio_summary.csv")

```

```
P:
0    115.0000
1    115.5000
2    117.3125
3    119.6500
4    120.4500
5    122.3000
6    123.6000
7    123.8500
8    125.7125
9    124.7500
dtype: float64
```

```
r:
0         NaN
1    0.004348
2    0.015693
3    0.019925
4    0.006686
5    0.015359
6    0.010630
7    0.002023
8    0.015038
9   -0.007656
dtype: float64
```

```
Y has the biggest standard deviation: 0.039977.
False    124.750000
True     121.046875
dtype: float64
```

```
Final dataframe:
   day  portfolio_value  portfolio_return
0    1         115.0000                NaN
1    2         115.5000            0.004348
2    3         117.3125            0.015693
3    4         119.6500            0.019925
4    5         120.4500            0.006686
5    6         122.3000            0.015359
6    7         123.6000            0.010630
7    8         123.8500            0.002023
8    9         125.7125            0.015038
9   10         124.7500           -0.007656
```

Task 4. (climate)

- Load the dataset from the csv file `climate.csv`.
- Compute a heat index HI using a formula:

$$HI = T + 0.1 \cdot H \cdot (T - 14.4),$$

where $T =$ `temperature` , $H =$ `humidity` .

- Identify the five days with the highest heat index HI .
- For each weekday (`day_number`), compute:
 - Average temperature,
 - Total rainfall,
 - Average heat index.

Use `groupby()` .

- Determine which weekday has the highest average heat index.
- Check if temperature and humidity are correlated.

```
In [9]: clim_df=pd.read_csv("climate.csv")

HI=clim_df["temperature_c"]+0.1*clim_df["humidity"]*(clim_df["temperature_c"]-14.4)
clim_df["heat_index"]=HI
print(clim_df)

top5_heat=clim_df['heat_index'].nlargest(5).index
print(f"\nDays with the highest heat index: {list(top5_heat)}.\n")

gr_clim_weekday=clim_df.groupby(clim_df["day_number"])

avg_temp=gr_clim_weekday["temperature_c"].mean()
total_rain=gr_clim_weekday["rain_mm"].sum()
avg_heat_index=gr_clim_weekday["heat_index"].mean()
print(avg_temp, total_rain, avg_heat_index)

max_heat_weekday=gr_clim_weekday["heat_index"].mean().idxmax()
print(f"\nWeekday nr {max_heat_weekday} has the highest average heat index.")
```

```
corr=clim_df["temperature_c"].corr(clim_df["humidity"])  
print(f"\nCorrelation is equal to ~{round(corr,5)}, which means those variables are barely correlated.")
```


	date	day_number	temperature_c	humidity	rain_mm	heat_index
0	2025-07-01	2	32	44	0.0	109.44
1	2025-07-02	3	31	55	NaN	122.30
2	2025-07-03	4	20	60	NaN	53.60
3	2025-07-04	5	33	77	0.0	176.22
4	2025-07-05	6	29	45	5.0	94.70
5	2025-07-06	7	32	40	NaN	102.40
6	2025-07-07	1	30	61	0.0	125.16
7	2025-07-08	2	30	73	NaN	143.88
8	2025-07-09	3	33	73	5.0	168.78
9	2025-07-10	4	29	65	0.0	123.90
10	2025-07-11	5	22	77	0.0	80.52
11	2025-07-12	6	24	61	5.0	82.56
12	2025-07-13	7	23	57	5.0	72.02
13	2025-07-14	1	24	63	NaN	84.48
14	2025-07-15	2	22	69	0.0	74.44
15	2025-07-16	3	34	65	5.0	161.40
16	2025-07-17	4	30	40	10.0	92.40
17	2025-07-18	5	24	72	10.0	93.12
18	2025-07-19	6	21	47	5.0	52.02
19	2025-07-20	7	21	59	0.0	59.94
20	2025-07-21	1	23	71	5.0	84.06
21	2025-07-22	2	22	60	10.0	67.60
22	2025-07-23	3	25	68	5.0	97.08
23	2025-07-24	4	26	49	10.0	82.84
24	2025-07-25	5	29	54	NaN	107.84
25	2025-07-26	6	20	59	0.0	53.04
26	2025-07-27	7	26	78	5.0	116.48
27	2025-07-28	1	23	64	0.0	78.04
28	2025-07-29	2	25	59	5.0	87.54
29	2025-07-30	3	32	61	NaN	139.36

Days with the highest heat index: [3, 8, 15, 7, 29].

day_number

1 25.00
2 26.20
3 31.00
4 26.25
5 27.00
6 23.50
7 25.50

Name: temperature_c, dtype: float64 day_number

1 5.0
2 15.0
3 15.0

```

4      20.0
5      10.0
6      15.0
7      10.0
Name: rain_mm, dtype: float64 day_number
1      92.935
2      96.580
3     137.784
4      88.185
5     114.425
6      70.580
7      87.710
Name: heat_index, dtype: float64

```

Weekday nr 3 has the highest average heat index.

Correlation is equal to ~ -0.12482 , which means those variables are barely correlated.

Task 5. (epidemic)

Track daily infections in different regions.

- Load the dataset from the csv file `epidemic.csv`.
- Count how many missing values are present in each column. Drop the rows containing missing values.
- Identify which region is in the majority and which is in the minority.
- Add a new column `infection_rate` using the formula:

$$I_t = \frac{\text{new_cases}}{\text{population}} \cdot 100.$$

- Create a new column `severity` based on `infection_rate`:
 - If the `infection_rate < 0.5` put `'Low'`,
 - If the `infection_rate < 1,5` put `'Moderate'`,
 - else `'High'`.
- Compute the sum of new cases for each region.
- Add a new column `growth_rate` with a weekly growth rate per region:

$$g_t = \frac{\text{new_cases}_t - \text{new_cases}_{t-7}}{\text{new_cases}_{t-7}} \times 100$$

Hint: use function `pct_change()` .

- Filter regions with `growth_rate > 10%` to identify high-risk regions and create the column `alert` in which put `1` if `growth_rate > 10%` and `0` otherwise.
- Count how many high-risk regions are present in the dataset.

```
In [12]: raw_epi_df=pd.read_csv("epidemic.csv")
print(raw_epi_df.head())

print(f"\nNumber of missing values:\n{raw_epi_df.isnull().sum()}")
clean_epi_df=raw_epi_df.dropna().copy() #it gets rid of the warning after creating infection_rate column

print("\nWe can see that the data from North and South are in a majority (we have the most records from there), whereas East and West are in a minority")
print(clean_epi_df.groupby("region")["day"].count())
#i hope this is what you meant by "in majority"/"in minority"

clean_epi_df["infection_rate"]=clean_epi_df["new_cases"]/clean_epi_df["population"]*100

def how_severe(rate):
    if rate<0.5:
        return("Low")
    if rate<1.5:
        return("Moderate")
    return("High")

clean_epi_df["severity"]=clean_epi_df["infection_rate"].apply(how_severe)

print("\nSum of new cases in each region:")
print(clean_epi_df.groupby("region")["new_cases"].sum())

clean_epi_df["growth_rate"]=clean_epi_df["infection_rate"].pct_change(periods=7)
clean_epi_df["alert"]=(clean_epi_df["growth_rate"]>0.1)*1
print(f"\nFinal dataframe:\n{clean_epi_df}")

print(f'\nDays with each alert:\n{clean_epi_df.groupby("alert")["day"].count()}')
```

	region	day	new_cases	population
0	North	2025-08-01	7.0	1626
1	North	2025-08-02	7.0	1624
2	North	2025-08-03	13.0	1629
3	North	2025-08-04	12.0	1631
4	North	2025-08-05	11.0	1630

Number of missing values:

region	0
day	0
new_cases	71
population	0

dtype: int64

We can see that the data from North and South are in a majority (we have the most records from there), whereas East and West are in a minority:

region	
East	180
North	185
South	185
West	179

Name: day, dtype: int64

Sum of new cases in each region:

region	
East	1733.0
North	1860.0
South	1867.0
West	1818.0

Name: new_cases, dtype: float64

Final dataframe:

	region	day	new_cases	population	infection_rate	severity	\
0	North	2025-08-01	7.0	1626	0.430504	Low	
1	North	2025-08-02	7.0	1624	0.431034	Low	
2	North	2025-08-03	13.0	1629	0.798036	Moderate	
3	North	2025-08-04	12.0	1631	0.735745	Moderate	
4	North	2025-08-05	11.0	1630	0.674847	Moderate	
..	...	...	...	...	...	...	
795	West	2026-02-12	10.0	969	1.031992	Moderate	
796	West	2026-02-13	11.0	972	1.131687	Moderate	
797	West	2026-02-14	7.0	975	0.717949	Moderate	
798	West	2026-02-15	12.0	971	1.235839	Moderate	
799	West	2026-02-16	14.0	975	1.435897	Moderate	

growth_rate alert

```
0      NaN      0
1      NaN      0
2      NaN      0
3      NaN      0
4      NaN      0
..      ...      ...
795    0.000000      0
796    0.002058      0
797   -0.121410      0
798    0.003090      0
799    0.545983      1
```

[729 rows x 8 columns]

Days with each alert:

alert

0 424

1 305

Name: day, dtype: int64

Challenge (employee bonuses and performance analysis)

This task is not mandatory!

Scenario:

You have been given a dataset of employees with information about salaries, performance, and tenure. The dataset contains some missing values and needs cleaning before use. Your job is to perform various Pandas operations to prepare and analyze this data.

Do the following steps:

Step 1. Load the Dataset

- Load the dataset from the CSV file: `employees.csv`

Step 2. Cleaning

- Count how many **missing values** there are in each column.
- Drop employees with missing `PerformanceScore` (assume they skipped evaluation).

- For missing values in `BaseSalary` or `YearsAtCompany` , **fill with the median within each department.**
(Hint: use `groupby(...).transform("median")`)
 - Confirm that no missing values remain.
-

Step 3. Core Transformations and Logic

- Add a new column `Bonus%` with the following rules:
 - If **PerformanceScore** ≥ 9 $\rightarrow$ `Bonus = 20% * BaseSalary`
 - If **7** $\leq$ **PerformanceScore** < 9 $\rightarrow$ `Bonus = 10% * BaseSalary`
 - Else $\rightarrow$ `Bonus = 0`
- Compute total pay:

$$\text{TotalPay} = \text{BaseSalary} + \text{Bonus}$$

Step 4. Descriptive Statistics

1. Compute department-wise:
 - Mean and median `BaseSalary`
 - Average `Bonus%`
 - Employee counts
 2. Find the **top 5 employees by TotalPay**.
 3. Identify:
 - Department with the **highest pay variability** (largest std. deviation of salaries)
 - Department with the **lowest average PerformanceScore**
 4. Compute the **correlation** between `BaseSalary` and `PerformanceScore` .
-

In []: